VISION

Observability Platform

Official Technical Documentation • v3.2 • 2025

Metrics · Traces · Logs · Alerting · Dashboards

4.2M+	22:1	800K/s	2GB/s
Queries/sec	Compression Ratio	Trace Ingest	Log Throughput

Vision is the next-generation unified observability platform that combines high-performance metrics storage, distributed tracing, and structured log management into a single cohesive system. Built from the ground up with VictoriaMetrics as its storage backbone, Vision delivers industry-leading query performance, cost efficiency, and operational simplicity at any scale.

Table of Contents

Chapter 1 – Introduction to the Vision Platform

1.1 What is Vision?

Vision is a **unified, cloud-native observability platform** purpose-built for modern distributed systems. Unlike traditional monitoring solutions that treat metrics, traces, and logs as separate concerns requiring separate stacks, Vision provides a single control plane that ingests, stores, correlates, and visualises all three observability signals with zero data silos.

At its core, Vision replaces the aging Prometheus TSDB storage engine with **VictoriaMetrics**, a high-performance time-series database engineered for compressibility, horizontal scalability, and low operational overhead. This architectural decision alone delivers a 22:1 compression ratio compared to Prometheus's 3:1, translating directly into storage cost savings of up to 85% at scale.

> ✔ Vision is fully compatible with existing Prometheus exporters, Grafana dashboards, and PromQL queries. Migration from Prometheus requires zero changes to your instrumentation code.

Core Capabilities:

- **Metrics:** High-throughput ingestion via VictoriaMetrics with native PromQL + VQL support
- **Tracing:** OpenTelemetry-native distributed tracing with adaptive sampling and tail-based sampling
- **Logs:** Petabyte-scale structured log storage via VictoriaLogs with full-text search and LogQL+
- **Correlation:** First-class cross-signal linking — jump from a metric spike to the causative trace to the error log in two clicks
- **Alerting:** Multi-dimensional SLO alerting with burn-rate windows, anomaly detection, and PagerDuty/OpsGenie integration
- **Dashboards:** Embedded Grafana-compatible visualisation layer with pre-built runbooks
- **Security:** mTLS, RBAC, audit logs, SSO via OIDC/SAML, and SOC 2 Type II compliance

1.2 Core Philosophy & Design Principles

Vision was designed around four guiding principles that distinguish it from legacy observability stacks:

Cardinality Without Fear

Legacy systems like Prometheus break under high-cardinality labels. Vision's VictoriaMetrics core handles billions of unique time series without degradation, enabling teams to instrument freely — add pod, version, region, and customer-id labels without planning around TSDB head-block memory limits.

Operational Simplicity at Scale

A single Vision binary can replace an entire Prometheus + Thanos + Alertmanager + Loki stack. Vision's cluster mode auto-shards ingestion and storage, eliminating the need for complex Thanos sidecar architectures, object-store compactors, and store gateways.

OpenTelemetry First

Vision treats OpenTelemetry as the universal instrumentation standard. All three signal types are ingested via OTLP (HTTP and gRPC), ensuring vendor-neutral instrumentation and future-proof observability.

Intelligent Correlation

Observability data is only valuable when correlated. Vision's Unified Correlation Engine automatically links trace exemplars to metric time-windows and enriches log entries with trace context, enabling sub-minute root cause analysis workflows.

1.3 Key Differentiators

Feature	Vision	Prometheus + Thanos	Datadog
Storage Engine	VictoriaMetrics (22:1 compression)	TSDB (3:1 compression)	Proprietary cloud
High Cardinality	Native — billions of series	Limited by TSDB head block	Cost-prohibitive at scale
Traces	Native OTLP, tail sampling	Via Jaeger/Tempo (separate)	Native APM (expensive)
Logs	VictoriaLogs (integrated)	Loki (separate stack)	Native (expensive)
Query Language	VQL + PromQL + LogQL+	PromQL only	DQL (proprietary)
Cross-Signal Correlation	Native, automatic	Manual, complex	Partial (paid tiers)
Deployment	Single binary or k8s operator	Multiple components	SaaS only
Pricing Model	Open-core, free OSS tier	Open source	$$$$ per host/metric
Global Long-Term Storage	Native cluster mode	Thanos/Cortex required	Managed, expensive
PromQL Compatibility	Full + extensions	Full	Partial

1.4 Architecture Overview

Vision's architecture is organised into three planes: the **Ingestion Plane**, the **Storage Plane**, and the **Query Plane**. All three planes can be deployed as a single monolithic binary for small deployments (up to ~1 million active series) or as independently scalable microservices in cluster mode.

Ingestion Plane	Storage Plane	Query Plane
• vmagent (scraping) • OTLP receiver • Prometheus remote-write • Syslog/FluentBit • Kafka consumer	• VictoriaMetrics (metrics) • VictoriaLogs (logs) • ClickHouse traces store • S3-compatible long-term • Local NVMe hot tier	• vmselect cluster • Trace query API • LogQL+ engine • Correlation engine • Grafana data-source

Figure 1.1 – Vision logical architecture planes

1.5 Supported Ecosystems

- **Kubernetes:** Helm chart, Operator CRDs, auto-discovery of pods and services
- **AWS:** CloudWatch Metrics Streams, ECS service discovery, EKS node exporter bundles
- **GCP:** Cloud Monitoring bridge, GKE autopilot support, BigQuery log sink
- **Azure:** Azure Monitor bridge, AKS discovery, Event Hub log ingest
- **On-Premises:** Bare-metal Linux, VMware, and Hyper-V via systemd units
- **OpenTelemetry Collector:** Drop-in exporter — no SDK changes required
- **Existing Prometheus Exporters:** node_exporter, kube-state-metrics, blackbox_exporter, mysqld_exporter, and 800+ community exporters

Chapter 2 – Why Vision? The Competitive Advantage

2.1 Storage Engine: VictoriaMetrics vs Prometheus TSDB

The most fundamental architectural difference between Vision and Prometheus is the storage engine. Prometheus uses its own custom TSDB, which was revolutionary in 2016 but has shown significant limitations at scale. Vision replaces TSDB with **VictoriaMetrics**, purpose-built for efficiency and scale.

Compression & On-Disk Size

Prometheus TSDB achieves approximately 1.3–3 bytes per sample using Gorilla-based delta-of-delta encoding. VictoriaMetrics applies a multi-stage compression pipeline — value encoding, delta-of-delta, bit-packing, and ZSTD — achieving **0.4–0.8 bytes per sample** in real-world workloads, a 3–7× reduction.

Metric	Prometheus TSDB	VictoriaMetrics (Vision)	Improvement
Bytes/sample (avg)	1.3 – 3.0	0.4 – 0.8	3–7× smaller
1B samples storage	~3 TB	~650 GB	78% less disk
Ingestion rate (single)	~200K samples/s	~4.2M samples/s	21× faster
Query latency (p99)	850 ms	95 ms	9× faster
High-cardinality series	Degrades >10M	Stable at 50B+	5000× more series
Memory per 1M series	~6 GB	~1.2 GB	5× less memory
Head block size	Unbounded growth	Fixed + optimised	Predictable ops
Compaction overhead	High (periodic)	Background, low	Reduced operational load

Table 2.1 – Storage engine benchmark comparison (1-node, 32-core, 64GB RAM)

2.2 Scalability & Performance Benchmarks

Vision has been benchmarked against Prometheus, Thanos, Cortex, and Mimir under identical hardware conditions. The following results were produced on a 3-node cluster (each node: 32 vCPU, 128 GB RAM, NVMe SSD) using the TSBS benchmark suite extended with Vision's test harness.

Test	Vision	Prometheus+Thanos	Cortex	Mimir
Write throughput	4.2M/s	0.9M/s	1.8M/s	2.1M/s
Active series	50B	10M	1B	5B
Range query p50	12 ms	210 ms	95 ms	72 ms
Range query p99	95 ms	850 ms	420 ms	310 ms
Instant query p50	3 ms	45 ms	22 ms	18 ms

Test	Vision	Prometheus+Thanos	Cortex	Mimir
Disk usage / 1B pts	650 GB	3 TB	1.2 TB	900 GB
CPU usage (ingest)	12 cores	28 cores	22 cores	18 cores
RAM usage (ingest)	18 GB	64 GB	40 GB	32 GB
Cold query latency	55 ms	12 s	2.1 s	1.8 s
HA complexity	Low	Very High	High	Medium

2.3 Cost of Ownership Analysis

Total cost of ownership (TCO) is driven by three factors: **compute**, **storage**, and **engineering effort**. Vision reduces all three. The following analysis compares a hypothetical organisation running 500 million active time series with 90-day retention.

Cost Driver	Prometheus Stack	Vision	Saving
Storage (90-day, S3)	$14,400/mo	$3,100/mo	78%
EC2 compute (cluster)	$8,200/mo	$2,100/mo	74%
Engineering FTEs	2.5 SREs	0.5 SREs	80%
Thanos/Cortex infra	$3,600/mo	$0	100%
Total monthly (est.)	~$26,200/mo	~$5,200/mo	80%

★ A Fortune 500 e-commerce customer migrated from Prometheus+Thanos to Vision and reduced their observability infrastructure spend from $310,000/year to $61,000/year — a saving of $249,000 annually while simultaneously improving query performance by 9×.

2.4 Operational Simplicity

One of Vision's most underappreciated advantages is the radical reduction in operational complexity. A production-grade Prometheus stack typically requires:

- Prometheus servers (sharded)
- Thanos sidecars per Prometheus replica
- Thanos Querier and Query Frontend
- Thanos Store Gateway
- Thanos Compactor
- Alertmanager cluster
- Loki for logs (separate stack)
- Jaeger or Tempo for traces (separate stack)

- Object store (S3/GCS)
- Grafana with multiple data sources

Vision replaces this 10-component architecture with:

- vmagent (ingestion, scraping, remote-write)
- vminsert + vmselect + vmstorage (metrics cluster)
- VictoriaLogs (log storage and query)
- Vision Trace Store (ClickHouse-backed)
- vmalert (alerting, recording rules)
- Vision UI (unified dashboard, no Grafana required)

> ■ 6 components vs 10. Every component Vision removes is a component you never have to upgrade, patch, monitor, or wake up at 3 AM to debug.

Chapter 3 – Vision vs Prometheus: Detailed Comparison

3.1 Metrics Collection & Scraping

Both Vision and Prometheus follow the pull-based scraping model where the platform periodically fetches metrics from instrumented targets. However, Vision's **vmagent** component extends this model significantly.

Capability	Vision (vmagent)	Prometheus
Max scrape targets	Millions (sharded)	~30K per instance
Scrape protocol	HTTP, HTTPS, gRPC	HTTP, HTTPS
Remote write (push)	Native, multi-target	Remote write v1/v2
OTLP push receiver	Native	Experimental only
Target discovery	50+ SD mechanisms	25+ SD mechanisms
Relabelling	VMRelabelling (extended)	Prometheus relabelling
Scrape caching	Built-in	Not available
Stateful scrape failover	Automatic	Manual Thanos shard
Stream parsing	Yes (low memory)	No (full parse in RAM)
Persistent scrape queue	Yes (WAL-backed)	No
Push-based ingest (Kafka)	Native connector	Not available
OpenMetrics 1.0	Full support	Full support

3.2 Storage & Retention

Prometheus storage is designed for short-term retention (days to a few weeks) with TSDB local disk. Long-term storage requires third-party solutions like Thanos, Cortex, or Mimir. Vision provides **native long-term storage** up to years, with automatic tiering between NVMe (hot), HDD (warm), and S3-compatible object storage (cold).

- **Hot tier (0–7 days):** NVMe SSD, fastest query performance, full resolution
- **Warm tier (7–90 days):** HDD or cheaper SSD, automatic downsampling to 1-minute resolution
- **Cold tier (90 days – 5 years):** S3/GCS/Azure Blob, 5-minute resolution, query-on-demand
- **Archive tier (5+ years):** Glacier/Nearline, 1-hour resolution, manual retrieval

Prometheus with Thanos requires manual configuration of all these tiers. Vision automates tiering via a single storage.tiers configuration block. Queries automatically fan out across tiers and merge results transparently.

3.3 Query Language: VQL vs PromQL

Vision Query Language (VQL) is a strict superset of PromQL. Every valid PromQL query is a valid VQL query. VQL adds the following extensions:

- **WITH expressions:** Named sub-expressions for query reuse (like SQL CTEs)
- **keep_metric_names modifier:** Preserve metric names through transformations
- **aggr_over_time functions:** Extended rollup functions (median_over_time, mode_over_time, zscore_over_time)
- **Implicit subqueries:** Automatic range vector deduction from step interval
- **Streaming aggregation:** Continuous query evaluation for real-time dashboards
- **Cross-series joins:** Lookup joins for enriching metrics with metadata

VQL Example – WITH Expression:

```
WITH ( error_rate(job) = sum(rate(http_requests_total{job=job, status=~'5..'}[5m])) /
sum(rate(http_requests_total{job=job}[5m])), slow_rate(job) =
sum(rate(http_request_duration_seconds_bucket{job=job, le='0.5'}[5m])) /
sum(rate(http_requests_total{job=job}[5m])) ) error_rate('api-gateway') > 0.01 OR
slow_rate('api-gateway') < 0.95
```

The same query in PromQL would require copy-pasting sub-expressions four times, making it fragile and hard to maintain.

3.4 High Availability & Clustering

HA Aspect	Vision	Prometheus+Thanos
Multi-replica writes	Native deduplication	Thanos dedup query layer
Horizontal ingest scale	vminsert (stateless, auto)	Sharding + federation (manual)
Horizontal query scale	vmselect (stateless, auto)	Thanos Query Frontend
Data replication	Configurable replica factor	Object store dependent
Failover time	<5 seconds	>30 seconds (Thanos restart)
Compaction conflicts	Handled automatically	Single compactor (SPOF)
Schema migrations	Zero-downtime	Requires restart
Rolling upgrades	Supported natively	Complex, error-prone

3.5 Remote Write & Federation

Vision's vmagent supports Prometheus Remote Write v1 and v2 protocols as both producer and consumer, making it a drop-in replacement in existing pipelines. Additionally, Vision introduces **VisionStream** — a proprietary high-throughput protocol with native compression that achieves 3× higher

remote-write throughput than Prometheus Remote Write v2.

- Fan-out remote write to multiple destinations simultaneously (e.g., Vision cluster + Prometheus backup)
- Configurable per-destination label filtering and relabelling
- Persistent on-disk queue with configurable size — no data loss during destination outages
- Automatic retry with exponential backoff
- mTLS authentication per destination
- Compression: snappy (default), zstd (Vision destinations)

Chapter 4 – Metrics: Deep Dive

4.1 Metric Types & Instrumentation

Vision supports all four standard Prometheus metric types plus three Vision-native extensions. All types are fully compatible with the OpenMetrics 1.0 specification.

Type	Description	Use Cases	Vision Extension
Counter	Monotonically increasing value	Requests, errors, bytes sent	rate_counter (rate-aware)
Gauge	Arbitrary float value (up or down)	Memory usage, queue length	—
Histogram	Sampled observations with buckets	Latency, payload size	Sparse histograms (native)
Summary	Pre-computed quantiles client-side	Client-reported SLO metrics	—
StateSet	Enum-like set of boolean states	Circuit breaker state	Vision native
Info	Constant labels (metadata metric)	Service version, build info	Vision native
GaugeHistogram	Gauge that tracks histogram	Queue wait time distribution	Vision native

Instrumentation Example – Go SDK:

```
import ( "github.com/vision-io/vision-go" ) var ( httpDuration =
vision.NewHistogram(vision.HistogramOpts{ Name: "http_request_duration_seconds", Help:
"HTTP request latency", Buckets: vision.ExponentialBuckets(0.001, 2, 15),
NativeHistogram: vision.NativeHistogramOpts{MaxZeroThreshold: 0.001}, }) requestsTotal =
vision.NewCounterVec(vision.CounterOpts{ Name: "http_requests_total", Help: "Total HTTP
requests", }, []string{"method", "status", "handler"}) )
```

4.2 VictoriaMetrics Storage Internals

VictoriaMetrics uses a log-structured merge (LSM) approach optimised for time-series access patterns. Incoming data is written to an in-memory raw row cache (rawRowsCache) and periodically flushed to on-disk parts. Parts are immutable and periodically merged in the background.

- **rawRowsCache:** In-memory ring buffer (configurable, default 512MB), absorbs write bursts
- **inmemoryPart:** Sorted in-memory part, holds unflushed rows
- **On-disk parts:** Immutable sorted files with columnar layout — timestamps and values stored separately
- **Merge:** Background goroutines continuously merge small parts into larger parts, similar to LSM compaction
- **Index:** Inverted index for label lookups (metric name, labels → time series IDs)
- **Retention:** Parts older than retention are atomically deleted at midnight UTC

✔ VictoriaMetrics does NOT use TSDB's head block / block structure. This eliminates the compaction stalls that Prometheus users experience every 2 hours as TSDB flushes head blocks to disk.

4.3 Ingestion Pipeline

The Vision ingestion pipeline handles multiple protocols concurrently. All protocols are parsed by a common normalisation layer that produces internal MetricRow objects before storage.

Protocol	Port (default)	Max Throughput	Notes
Prometheus Remote Write v1	8480	1.2M rows/s	Fully compatible
Prometheus Remote Write v2	8480	2.1M rows/s	Native histograms support
VisionStream (proprietary)	8481	4.2M rows/s	Best performance
OTLP/gRPC	4317	900K rows/s	OpenTelemetry standard
OTLP/HTTP+JSON	4318	600K rows/s	Human-readable debugging
Graphite plaintext	2003	800K rows/s	Legacy compatibility
InfluxDB line protocol	8086	1.1M rows/s	InfluxDB migration
OpenTSDB HTTP API	4242	700K rows/s	OpenTSDB migration
Kafka consumer	N/A	3.0M rows/s	Async, durable ingest
AWS CloudWatch Streams	N/A	500K rows/s	Managed AWS integration

4.4 VQL Query Engine Reference

Key VQL functions and their usage:

Function	Description
`rate(v[d])`	Computes per-second average rate of change over duration d. Use for counters.
`irate(v[d])`	Instant rate using the last two data points. Better for dashboards with fine resolution.
`increase(v[d])`	Total increase of counter over duration d. Equivalent to rate(v[d]) * d.
`rollup(v[d])`	Returns min/max/avg triple for gauge metrics over duration. Vision-only.
`median_over_time(v[d])`	Median of gauge values over duration. More robust than avg_over_time.
`zscore_over_time(v[d])`	Z-score normalisation over time window. Useful for anomaly detection.
`histogram_quantile(phi, le)`	Estimate quantile from histogram buckets. Same as PromQL.
`histogram_share(le, v)`	Fraction of histogram observations below threshold le. Vision-only.

Function	Description
`topk_max(k, v[d])`	Top-K series by their maximum value over duration. Vision-only.
`with(expr, ...)`	Named sub-expression. Evaluated once and reused. Vision-only.
`label_transform(v, l, r, rep)`	Regex transform on label value. Vision-only.
`aggr_over_time(funcs, v[d])`	Apply multiple aggregation functions in one pass. Vision-only.

4.5 Downsampling & Rollups

Vision implements automatic multi-resolution downsampling. When data ages out of the hot tier, the downsampler runs and produces pre-aggregated rollups at configurable resolutions. This dramatically reduces storage and improves long-range query performance.

- **Raw:** Full resolution (typically 15s or 30s scrape interval) — hot tier only
- **5-min rollup:** min, max, avg, count stored per 5-minute window — warm tier
- **1-hour rollup:** min, max, avg, count stored per hour — cold tier
- **1-day rollup:** min, max, avg, count stored per day — archive tier

Rollup metadata is transparent to the query engine — VQL automatically selects the correct resolution based on the requested time range and step interval.

4.6 Alerting & Recording Rules

Vision uses **vmalert** for rule evaluation. vmalert is a highly scalable, stateless rule evaluation engine that supports PromQL and VQL rules in YAML format (100% Prometheus-compatible).

Example Alert Rule:

```
groups: - name: api-gateway-slos interval: 30s rules: - alert: HighErrorRate expr: |
WITH (job = "api-gateway") sum(rate(http_requests_total{job=job, status=~"5.."}[5m])) /
sum(rate(http_requests_total{job=job}[5m])) > 0.01 for: 2m labels: severity: critical
team: platform annotations: summary: "Error rate {{ $value | humanizePercentage }} on {{
$labels.job }}" runbook_url: "https://wiki.example.com/runbooks/high-error-rate"
vision_dashboard: "https://vision.example.com/d/api-gateway?from=now-1h"
```

4.7 Vision Metrics vs Prometheus vs Thanos vs Cortex

Capability	Vision	Prometheus	Thanos	Cortex
Long-term storage	Native	External (Thanos)	Yes	Yes
HA without Thanos	Yes	No	Yes	Yes

Capability	Vision	Prometheus	Thanos	Cortex
High cardinality	Excellent	Poor	Moderate	Good
Multi-tenancy	Full RBAC	No	Basic	Full
Query federation	Native global	Manual	Yes	Yes
VQL extensions	Yes	No	No	No
Native histograms	Yes	Experimental	Partial	Partial
Downsampling	Automatic	No	Manual	Manual
Rule evaluation scale	Millions/min	Thousands/min	Thousands/min	Millions/min
Deployment complexity	Low	Low	Very High	High
Cloud-native operator	Yes	Partial	Yes	Yes
OpenSource license	Apache 2.0	Apache 2.0	Apache 2.0	Apache 2.0

Chapter 5 – Distributed Tracing: Deep Dive

5.1 Tracing Concepts & Terminology

Distributed tracing is the practice of tracking a single request as it propagates through multiple services in a distributed system. Vision's tracing subsystem is built on the **OpenTelemetry specification** and provides first-class support for the W3C TraceContext propagation standard.

Term	Definition
Trace	The end-to-end record of a request across all services. Identified by a globally unique 128-bit TraceID.
Span	A single unit of work within a trace. Has a start time, end time, operation name, and attributes.
SpanContext	Propagation metadata: TraceID, SpanID, TraceFlags, TraceState. Transmitted via HTTP headers or gRPC metadata.
Parent Span	The span that created the current span. Root spans have no parent.
Trace Attributes	Key-value metadata attached to spans (e.g., http.method, db.statement, error.message).
Span Events	Point-in-time annotations within a span (e.g., 'cache miss', 'retry attempt 2').
Span Links	Explicit links between spans across different traces (e.g., async processing).
Baggage	User-defined key-value pairs propagated throughout the trace. Used for correlation IDs.
Exemplar	A specific trace ID attached to a metric data point, enabling metric-to-trace correlation.
Sampling	The decision of whether to record a trace. Critical for cost control at high request volumes.

5.2 Vision Trace Architecture

Vision's trace subsystem consists of three components: the **Vision Trace Collector** (VTC), the **Trace Storage Engine** (ClickHouse-backed), and the **Trace Query API**.

- **Vision Trace Collector (VTC):** Stateless OTLP receiver with built-in tail-based sampling. Can be deployed as a DaemonSet (per-node) or Deployment (centralised). Buffers spans in-memory and flushes to ClickHouse in batches of 50,000 spans every 5 seconds.
- **Trace Storage (ClickHouse):** Columnar storage optimised for trace queries. Spans are stored in a flat table with pre-computed indexed columns: trace_id, span_id, service_name, operation_name, start_time, duration_ns, status_code, and up to 50 attribute key-value pairs per span.
- **Trace Query API:** REST API for trace retrieval, search, and aggregation. Supports TraceQL (OpenTelemetry's trace query language) and Vision Trace Extensions (VTE).

Ingest Throughput:

800K/s	30 days	~180 GB	320 ms
Span Ingest Rate	Trace Retention	Storage / 1B spans	Query Latency p99

5.3 OpenTelemetry Integration

Vision is 100% OpenTelemetry-native. Any application instrumented with the OpenTelemetry SDK can send traces to Vision with zero code changes — only the OTLP endpoint needs to be configured.

Python Instrumentation Example:

```
from opentelemetry import trace from opentelemetry.sdk.trace import TracerProvider from opentelemetry.sdk.trace.export import BatchSpanProcessor from opentelemetry.exporter.otlp.proto.grpc.trace_exporter import OTLPSpanExporter provider = TracerProvider(resource=resource) provider.add_span_processor( BatchSpanProcessor( OTLPSpanExporter(endpoint="https://vision.example.com:4317"), ) ) trace.set_tracer_provider(provider) tracer = trace.get_tracer("my-service") with tracer.start_as_current_span("process-order") as span: span.set_attribute("order.id", order_id) span.set_attribute("customer.tier", "premium") result = process_order(order_id) span.set_attribute("order.total_usd", result.total)
```

5.4 Trace Storage & Retention

Traces are stored in a sharded ClickHouse cluster with a configurable retention policy. Unlike Jaeger's Cassandra or Elasticsearch backends — which require careful capacity planning and suffer from write amplification — ClickHouse's columnar format provides excellent compression and fast analytical queries over trace data.

Aspect	Vision	Jaeger (Cassandra)	Jaeger (Elasticsearch)	Tempo
Storage backend	ClickHouse	Cassandra	Elasticsearch	Object storage
1B spans disk size	~180 GB	~800 GB	~1.2 TB	~120 GB
Write throughput	800K spans/s	120K spans/s	80K spans/s	200K spans/s
Search by attribute	Fast (indexed)	Slow (full scan)	Fast (inverted idx)	No (trace-ID only)
Aggregation queries	Native SQL	Limited	Elasticsearch DSL	No
Dependency graph	Automatic	Manual	Manual	Manual
Service map	Real-time	Periodic	Periodic	No
Retention config	Per-service TTL	Cassandra TTL	ILM policies	Object store lifecycle
Multi-tenancy	Native RBAC	Keyspace isolation	Index isolation	Tenant prefix

5.5 Trace Querying & Visualisation

Vision supports **TraceQL** — the OpenTelemetry trace query language — plus Vision Trace Extensions (VTE) for aggregation, comparison, and anomaly queries.

TraceQL Example – Find slow database spans:

```
{ span.db.system = "postgresql" && duration > 500ms && status = error } |
select(resource.service.name, span.db.statement, duration)
```

VTE Aggregation – P99 latency by service pair:

```
SELECT parent_service, child_service, quantile(0.99, duration_ns) / 1e6 AS p99_ms,
count() AS call_count FROM vision_traces WHERE start_time >= now() - INTERVAL 1 HOUR AND
status_code != 0 GROUP BY parent_service, child_service ORDER BY p99_ms DESC LIMIT 20
```

5.6 Vision vs Jaeger vs Zipkin vs Tempo

Feature	Vision	Jaeger	Zipkin	Grafana Tempo
OTLP native	Yes	Yes (v2)	Via bridge	Yes
Tail-based sampling	Yes, built-in	External	No	Yes (external)
TraceQL support	Yes	No	No	Yes
Metrics correlation	Native exemplar	Manual	No	Grafana links
Log correlation	Native	Manual	No	Via Loki
Attribute search	Indexed, fast	Limited	Limited	No (ID only)
Service map	Automatic	Yes	Yes	No
UI quality	Excellent	Good	Basic	Via Grafana
Deployment	Integrated	Standalone	Standalone	Standalone
Metrics from spans	Yes (SpanMetrics)	No	No	SpanMetrics pipe
Anomaly detection	Built-in ML	No	No	No

5.7 Sampling Strategies

Sampling is essential for controlling the volume and cost of trace data. Vision supports four sampling strategies that can be combined:

Head-Based (Probabilistic)

Sampling decision made at trace root. Simple and low overhead. Recommended for development. Configurable per-service rate.

Head-Based (Rate Limiting)

Cap traces per second per service. Prevents runaway cardinality during traffic spikes.

Tail-Based (Error Traces)

Buffer spans for 30 seconds, then sample 100% of traces with at least one error span and 1% of successful traces. Vision's default for production.

Tail-Based (Latency-Aware)

Sample 100% of traces exceeding the P99 threshold, 10% of P90–P99 traces, and 1% of traces below P90. Captures all anomalies automatically.

Chapter 6 – Log Management: Deep Dive

6.1 Log Ingestion Pipeline

Vision ingests logs from any source via FluentBit, Logstash, the OpenTelemetry Collector, or direct OTLP push from applications. The ingestion pipeline handles parsing, structuring, enrichment, and routing before writing to VictoriaLogs.

Source	Integration Method
FluentBit (recommended)	DaemonSet deployment, tail log files, Kubernetes metadata enrichment
OpenTelemetry Collector	OTLP/gRPC or OTLP/HTTP from applications, structured logging
Logstash	Beats inputs, Grok parsing, filter pipelines
Syslog (UDP/TCP)	Legacy systems, network devices, RFC 5424 and RFC 3164
AWS CloudWatch Logs	Kinesis Firehose subscription filter → Vision HTTP endpoint
GCP Cloud Logging	Pub/Sub export → Vision Cloud Logging bridge
Kafka	High-throughput async ingest with guaranteed delivery
Direct HTTP API	curl / SDK push — JSON or newline-delimited JSON (NDJSON)

6.2 VisionLogs Storage (VictoriaLogs)

VisionLogs uses **VictoriaLogs** as its storage engine — a log management system built by the VictoriaMetrics team and deeply integrated into Vision's unified architecture. VictoriaLogs stores logs as structured JSON documents with inverted-index search, achieving remarkable compression and query performance at petabyte scale.

Key architectural properties:

- **Columnar + inverted index:** Each log field is stored in its own column with a full-text inverted index, enabling fast field-level filtering
- **Block compression:** ZSTD compression per block achieves 10:1 to 30:1 compression ratios on real log data
- **Stream concept:** Logs are organised into 'streams' (label sets, e.g., {app='nginx', env='prod'}) — identical to Loki's model
- **Pipeline filters:** Server-side log processing pipeline — filter, parse, extract, aggregate — without shipping to client
- **Multi-tenancy:** Tenant isolation via account ID label with per-tenant quotas and retention

★ VictoriaLogs stores 1 TB of raw logs as approximately 35–100 GB on disk. Elasticsearch typically stores 1 TB of raw logs as 1.5–3 TB (inverted index overhead). This means Vision stores logs at 15–85× less disk cost than Elasticsearch.

6.3 Log Query Language – LogQL+ Reference

Vision's log query language, **LogQL+**, is a strict superset of Grafana Loki's LogQL. All existing LogQL queries work unchanged. LogQL+ adds powerful extensions for structured log analysis and cross-signal correlation.

Basic LogQL+ Syntax:

```
# Stream selector (Loki-compatible) {app="api-gateway", env="prod"} # Filter pipeline
{app="api-gateway"} |= "error" | json | status_code >= 500 # Metric query (log-based)
sum by (status_code) ( rate({app="api-gateway"} | json | status_code > 0 [5m]) ) #
LogQL+ extension: field extraction + aggregation {app="api-gateway"} | json | extract
"duration_ms=" from message | quantile_over_time(0.99, dur [5m]) by (upstream)
```

LogQL+ Unique Extensions:

Extension	Description
`| extract`	Regex and pattern-based field extraction from unstructured log messages
`| decolorize`	Strip ANSI color codes from log messages before parsing
`| drop`	Remove fields from the log line (reduce bandwidth, hide PII)
`| keep`	Retain only specified fields
`| line_format`	Go template-based log line reformatting
`| label_format`	Rename or transform extracted label values
`quantile_over_time`	Compute quantiles over log-derived numeric fields
`first_over_time / last_over_time`	First or last non-null value in range
`correlate()`	Cross-signal join: attach metric or trace data to log lines
`stream_selector join`	Join two log streams on a common field

6.5 Vision Logs vs Loki vs Elasticsearch vs Splunk

Capability	Vision Logs	Grafana Loki	Elasticsearch	Splunk
Storage efficiency	Excellent (30:1)	Good (10:1)	Poor (1:2)	Poor (1:2)
Full-text search	Yes (inverted idx)	Limited	Yes	Yes
Structured field search	Yes (indexed)	Labels only	Yes	Yes

Capability	Vision Logs	Grafana Loki	Elasticsearch	Splunk
Query language	LogQL+	LogQL	KQL/DSL	SPL
Write throughput	2 GB/s	500 MB/s	300 MB/s	200 MB/s
Metrics from logs	Yes (native)	Yes	Via Elastic	Yes (expensive)
Trace correlation	Native	Via Grafana	Via APM plugin	Via Splunk APM
Alerting on logs	Native	Via Grafana	Watcher	Native
Multi-tenancy	Native RBAC	Basic	Full (licensed)	Full (expensive)
Schema-on-read	Yes	Yes	No (schema-on-write)	No
Cost	Low (OSS)	Low (OSS)	Medium-High	Very High
Kubernetes-native	Excellent	Good	Moderate	Poor
Regex in queries	Yes	Yes	Yes	Yes
Anomaly detection	Built-in	No	ML plugin	ML Toolkit ($$)

6.6 Log-Metric Correlation

Vision provides **automatic log-metric correlation** — when a metric alert fires, Vision can automatically surface the most relevant log lines from the same time window, service, and pod. This is powered by the Unified Correlation Engine described in Chapter 7.

- Click a metric spike in a dashboard → Vision automatically queries VisionLogs for log lines from the same service within ±5 minutes of the spike
- Log-derived metrics: parse numeric fields from logs and expose them as VictoriaMetrics gauge/counter metrics
- Alert correlation: when an alert fires, the notification includes relevant log excerpts
- Log volume as a metric: track log error rates per service as a time-series for alerting

Chapter 7 – Unified Correlation Engine

7.1 Metrics-Traces-Logs Linking

The Unified Correlation Engine (UCE) is Vision's crown jewel — a purpose-built subsystem that creates automatic bidirectional links between all three observability signals. Unlike other platforms that bolt on cross-signal navigation as an afterthought, UCE is a first-class architectural component designed from day one.

Correlation Dimensions:

From	To	Mechanism	Latency
Metric data point	Trace	Exemplar (trace_id, span_id in metric sample)	< 1ms
Metric alert	Logs	Time-window + service label join	< 100ms
Trace span	Metrics	SpanMetrics pipeline → VictoriaMetrics	Real-time
Trace span	Logs	trace_id propagation in log fields	< 50ms
Log line	Trace	trace_id field extraction → trace lookup	< 50ms
Log line	Metrics	log-derived metric materialization	Real-time

7.2 Exemplars Support

Exemplars are trace IDs attached to individual metric samples. When a histogram bucket records an observation, Vision attaches the current trace ID from the OpenTelemetry context. This creates a direct link from any point on a latency graph to the exact trace that produced that data point.

Vision's Grafana-compatible exemplar API serves exemplars alongside metric query results. In the Vision UI, exemplar dots appear on time-series graphs — clicking a dot opens the corresponding trace waterfall view instantly.

7.3 Root Cause Analysis Workflows

Typical RCA Workflow with Vision:

1. PagerDuty fires alert: *HighErrorRate* on service **order-service**
2. Vision alert notification includes direct link to pre-built dashboard filtered to the alert window
3. Dashboard shows error rate spike at 14:32 UTC — click the spike to see exemplars
4. Exemplar dot links to trace ID a3f9c2... — click to open trace waterfall
5. Trace shows span postgres.query took 8.2 seconds — 50× above P99 baseline
6. Vision automatically surfaces correlated logs from the same pod at 14:32 UTC
7. Logs show: FATAL: connection pool exhausted, max_connections=100 reached

8. Root cause identified: connection pool saturation → 8.2s query wait → cascading timeouts
9. Total time from alert to root cause: **4 minutes**

■ Without Vision's UCE, the same RCA would require manually switching between Grafana (metrics), Jaeger (traces), and Kibana (logs), correlating timestamps by hand — typically 30–90 minutes.

Chapter 8 – Alerting & Incident Management

8.1 Alert Rules Engine

Vision's alert rules engine (**vmalert**) evaluates VQL and LogQL+ expressions on a configurable interval. Rules are stored as YAML files and loaded dynamically — no restart required.

- Evaluation interval: configurable per group (default 30s, minimum 1s for critical rules)
- Concurrency: evaluates rule groups in parallel, each group sequentially
- State persistence: pending/firing/resolved state persisted in VictoriaMetrics — survives restarts
- Recording rules: materialise expensive query results as new time-series for fast dashboard queries
- Template functions: Go template rendering in alert labels and annotations with 40+ helper functions
- Inhibit rules: suppress low-priority alerts when higher-priority alerts are active
- Silences: time-window silencing via API or UI — no false positive noise during maintenance

8.2 Multi-Window, Multi-Burn-Rate SLOs

Vision implements the Google SRE book's multi-window, multi-burn-rate alerting strategy out of the box. Define your SLO once and Vision automatically generates the six alert rules required for critical (1h/5h windows) and warning (6h/3d windows) burn rates.

SLO Configuration Example:

```
apiVersion: vision.io/v1 kind: SLO metadata: name: api-gateway-availability spec: service: api-gateway target: 0.999 # 99.9% window: 30d indicator: ratio: good: metric: http_requests_total{status!~"5.."} total: metric: http_requests_total alerting: burnRateThreshold: 14.4 # Critical: 1h window burnRateThreshold5h: 6.0 # Critical: 5h window burnRateThreshold6h: 3.0 # Warning: 6h window burnRateThreshold3d: 1.0 # Warning: 3d window
```

8.3 Alert Routing & Notifications

Vision integrates with all major incident management platforms. Alert routing uses a tree-based matching engine (identical to Alertmanager's model) with label matchers.

Integration	Protocol	Features
PagerDuty	Events API v2	Severity mapping, auto-resolve, incident links
OpsGenie	REST API	Team routing, escalation, on-call schedule
Slack	Incoming Webhooks	Rich formatting, dashboard link, runbook URL
Microsoft Teams	Adaptive Cards	Structured alert cards, action buttons
Webhook	HTTP POST (JSON)	Custom integrations, arbitrary payloads

Integration	Protocol	Features
Email	SMTP	HTML templates, digest mode
VictoriaMetrics	Remote write	Alert state as metrics for meta-monitoring
Jira	REST API	Auto-create/resolve issues from alerts
ServiceNow	REST API	ITSM ticket creation with CI mapping

8.4 Anomaly Detection

Vision includes a built-in ML-powered anomaly detection engine (**VisionAD**) that automatically learns the baseline behaviour of each time series and generates dynamic thresholds. Unlike static threshold alerts that require constant tuning, VisionAD adapts to seasonality, business hours, and gradual growth.

- **Algorithms:** Seasonal decomposition (STL), Z-score with MAD, Isolation Forest for multivariate
- **Training window:** Configurable (default: 4 weeks of historical data)
- **Seasonality:** Automatic detection of daily and weekly patterns
- **Sensitivity:** Configurable sensitivity level (1–10 scale) per metric or metric group
- **Alert integration:** Anomaly scores exposed as VictoriaMetrics metrics — trigger alerts using standard VQL rules
- **Explainability:** Vision UI shows the expected band and the deviation magnitude for each anomaly

Chapter 9 – Deployment & Operations

9.1 Single-Node vs Cluster Mode

Vision is distributed as a single multi-binary package. The **vision-single** binary runs all components in a single process (ideal for development and small production deployments). The **vision-cluster** package runs each component independently for horizontal scalability.

Aspect	Single-Node	Cluster Mode
Binary	vision-single	vminsert + vmselect + vmstorage
Max active series	~100M	Unlimited (add nodes)
Max ingest rate	~4M samples/s	Linear with vminsert nodes
Replication	No	Configurable (default: 2)
High availability	No	Yes
Deployment time	< 5 minutes	~30 minutes (k8s operator)
Recommended for	Dev, small prod (<10M series)	Large prod (>10M series)

9.2 Kubernetes Deployment

The Vision Kubernetes Operator manages the full lifecycle of Vision components as Kubernetes Custom Resources. Installation via Helm chart takes under 10 minutes.

Quick Start:

```
# Add the Vision Helm repository helm repo add vision https://charts.vision.io helm repo update # Install Vision operator helm install vision-operator vision/vision-operator \ --namespace vision-system \ --create-namespace # Deploy a Vision cluster cat << EOF | kubectl apply -f - apiVersion: vision.io/v1 kind: VisionCluster metadata: name: prod spec: retentionPeriod: 90d replicationFactor: 2 storage: volumeClaimTemplate: spec: storageClassName: fast-ssd resources: requests: storage: 500Gi resources: requests: {cpu: "4", memory: "16Gi"} limits: {cpu: "16", memory: "64Gi"} EOF
```

9.3 Configuration Reference

Core configuration parameters for Vision single-node:

Flag	Default	Description
`-retentionPeriod`	`90d`	How long to retain data (supports h, d, w, y suffixes)
`-storageDataPath`	`/var/lib/vision`	Local directory for data storage

Flag	Default	Description
`-httpListenAddr`	`:8428`	HTTP API listen address
`-maxConcurrentInserts`	`16`	Max concurrent insert goroutines
`-memory.allowedPercent`	`60`	Percent of RAM allowed for caches
`-search.maxQueryDuration`	`30s`	Max time for a single query to execute
`-search.maxUniqueTimeseries`	`1000000`	Max unique time series per query response
`-replicationFactor`	`2`	Data replication factor (cluster mode)
`-dedup.minScrapeInterval`	`30s`	Deduplication interval for HA pairs
`-downsampling.period`	`30d:5m,180d:1h`	Downsampling schedules (retention:resolution pairs)

9.5 Security & RBAC

Vision implements a fine-grained Role-Based Access Control (RBAC) model with the following built-in roles. Custom roles can be defined with per-resource permissions.

Role	Metrics	Traces	Logs	Alerting	Config
Admin	R/W/D	R/W/D	R/W/D	R/W/D	R/W
Editor	R/W	R/W	R/W	R/W	R
Viewer	R	R	R	R	—
Alerts-Manager	R	—	—	R/W/D	—
Ingest-Only	W	W	W	—	—
Auditor	R	R	R	R	R (read-only)

- **Authentication:** Local users, LDAP, OIDC (Google, Okta, Auth0), SAML 2.0
- **Transport Security:** TLS 1.3 for all HTTP endpoints; mTLS for inter-component gRPC
- **Audit Logging:** All API calls logged with user, timestamp, IP, action, and resource
- **Secret Management:** Integration with Vault, AWS Secrets Manager, and Kubernetes Secrets
- **Compliance:** SOC 2 Type II, ISO 27001, GDPR data residency controls

Chapter 10 – SDK & API Reference

10.1 HTTP API

Endpoint	Method	Description	Compatible With
/api/v1/query	GET	Instant VQL/PromQL query	Prometheus, Grafana
/api/v1/query_range	GET	Range VQL/PromQL query	Prometheus, Grafana
/api/v1/series	GET	Find time series by matchers	Prometheus
/api/v1/labels	GET	List label names	Prometheus
/api/v1/label/<n>/values	GET	List values for label name	Prometheus
/api/v1/write	POST	Prometheus Remote Write ingest	Prometheus
/vision/stream/write	POST	VisionStream high-throughput ingest	Vision only
/otlp/v1/metrics	POST	OTLP metrics ingest (HTTP+Proto)	OpenTelemetry
/otlp/v1/traces	POST	OTLP traces ingest (HTTP+Proto)	OpenTelemetry
/otlp/v1/logs	POST	OTLP logs ingest (HTTP+Proto)	OpenTelemetry
/vision/traces/query	POST	TraceQL + VTE trace queries	Vision only
/vision/logs/query	GET	LogQL+ log queries	Loki-compatible
/vision/logs/query_range	GET	LogQL+ range queries	Loki-compatible
/api/v1/rules	GET	List loaded alert and recording rules	Prometheus
/api/v1/alerts	GET	List active alerts	Prometheus
/vision/slos	GET/POST	SLO management API	Vision only
/vision/anomalies	GET	Anomaly detection results	Vision only
/-/health	GET	Health check endpoint	Standard
/-/ready	GET	Readiness check (cluster aware)	Standard
/metrics	GET	Vision self-metrics (Prometheus fmt)	Prometheus

10.2 OpenTelemetry SDK Integration

Vision accepts data from any OpenTelemetry SDK without modification. The OTLP endpoint is compatible with all language SDKs. The following table shows the configuration required to point standard OTEL SDKs at Vision.

Language	Environment Variable / Config	Value
All (env vars)	OTEL_EXPORTER_OTLP_ENDPOINT	https://vision.example.com:4317
All (env vars)	OTEL_EXPORTER_OTLP_PROTOCOL	grpc (recommended) or http/protobuf
All (env vars)	OTEL_SERVICE_NAME	your-service-name
Go	otlptracegrpc.NewClient(...)	WithEndpoint("vision.example.com:4317")
Python	OTLPSpanExporter(endpoint=...)	endpoint="https://vision.example.com:4317"
Java	otel.exporter.otlp.endpoint	https://vision.example.com:4317
Node.js	OTLPTraceExporter({url:...})	url: 'https://vision.example.com:4318/v1/traces'
.NET	AddOtlpExporter(opt => ...)	opt.Endpoint = new Uri("...:4317")

10.3 Prometheus-Compatible Endpoints

Vision is a drop-in replacement for Prometheus as a Grafana data source. Configure Grafana with Vision's URL and select 'Prometheus' as the data source type — all existing dashboards and alert rules will work without modification.

Grafana Data Source Configuration:

```
# In Grafana UI: Configuration → Data Sources → Add data source Name: Vision Production
Type: Prometheus URL: http://vision.example.com:8428 Scrape interval: 30s # Optional:
Enable exemplars for metric-trace correlation Exemplars: - Internal Link: Vision Traces
Label name: trace_id URL: http://vision.example.com/explore?traceId=${__value.raw}
```

Summary – Why Vision is the Future of Observability

Vision represents a fundamental rethinking of observability infrastructure. By unifying metrics, traces, and logs into a single platform powered by VictoriaMetrics, Vision eliminates the operational complexity, cost, and data silos of legacy multi-stack architectures.

Key Takeaways:

- **22:1 compression** vs Prometheus TSDB's 3:1 — 78% less storage cost
- **4.2M samples/second** ingest rate — 21× faster than Prometheus
- **800K spans/second** trace throughput with sub-second query latency
- **2 GB/second** log ingestion with 30:1 compression vs Elasticsearch's 1:2
- **One platform** replacing Prometheus + Thanos + Loki + Jaeger + Alertmanager
- **Native correlation** enabling 4-minute RCA vs 30–90 minutes with siloed tools
- **Full Prometheus compatibility** — zero migration risk for existing instrumentation
- **80% TCO reduction** at scale for storage, compute, and engineering effort

■ Vision is open-core with a generous free tier (up to 10M active series, 30-day retention, single-node). Enterprise features (clustering, RBAC, SSO, long-term storage, anomaly detection) are available under the Vision Enterprise license. Get started at https://vision.io/docs

---